ORGNZ-06: Security Context

> **Series:** ORGNZ — Organize Data: Buckets, Segments, Security | **Notebook:** 6 of 10 | **Created:** January 2026 | **Last Updated:** 10/14/2026

Overview

If permissions on deployment-level attributes or bucket level are insufficient, Dynatrace allows you to set up **fine-grained permissions** by adding a `dt.security_context` attribute to your data. This enables record-level access control that scales beyond bucket limits.

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS environment with Grail enabled
Permissions	`storage:logs:read`, OpenPipeline configuration access
Knowledge	Completed ORGNZ-05 (Bucket-Level Access Control)
Data	Logs with `dt.security_context` field (or ability to configure)

Table of Contents

1. What is Security Context?
2. Security Context Values
3. Setting Security Context
4. IAM Policies with Security Context
5. Querying Security Context

6. Security Context Patterns
7. Security Context Best Practices
8. Entity Security Context

Learning Objectives

By the end of this notebook, you will:
- Understand the purpose and structure of security context
- Know how to assign security context via OpenPipeline
- Create IAM policies that use security context
- Design hierarchical security context patterns

What is Security Context?

Security context is a reserved field (`dt.security_context`) that enables custom authorization:

Aspect	Description
Field name	`dt.security_context`
Type	String or array of strings
Purpose	Custom authorization beyond standard attributes
Source	OpenPipeline, OneAgent, Kubernetes labels
Enforcement	IAM policies filter at query time

Why Use Security Context?

Scenario	Without Security Context	With Security Context
100 teams sharing data	Need 100 buckets	Single bucket, 100 context values
Dynamic team membership	Static bucket assignment	Update context via tags

Scenario	Without Security Context	With Security Context
Hierarchical access	Complex policy combinations	Hierarchical string encoding

Security Context Values

Simple Values

```
dt.security_context: "team-a"
dt.security_context: "finance"
dt.security_context: "sec-lvl-7"
```

Hierarchical Encoding

Encode hierarchy in a string for flexible access patterns:

```
dt.security_context: "department-A/team-1/project-x"
dt.security_context: "org1/finance/compliance"
dt.security_context: "region-us/aws-account-123/team-platform"
```

Array Values

Records can have multiple security context values:

```
dt.security_context: ["team-a", "project-x", "compliance"]
```

Setting Security Context

"Set the dt.security_context field as early as possible in the data pipeline, ideally at the source." Dynatrace lists the patterns in order of preference and is explicit about the order: *"At-source enrichment is always preferred over OpenPipeline-based enrichment for the security context."* A value set at the source is on the record before pipeline routing, so the record reaches the right pipeline and bucket from the start.

> **Corrected 09/18/2026.** Earlier versions of this notebook showed a `dynatrace.com/security-context` Kubernetes label and annotation, a YAML OpenPipeline processor, and an automatic host-group mapping. None of those is documented, and Dynatrace does not read that label or annotation. If you copied them, replace them with one of the patterns below.

1. At the source (preferred)

Source	How
OneAgent host tag	*"OneAgent host tags: Add dt.security_context= as a host tag during installation or via Deployment Status."* Applies to all telemetry from that host.
Process on shared infrastructure	Set `DT_TAGS="dt.security_context=<value>"` as an environment variable on the process.
Kubernetes	Namespace labels or annotations, mapped through Kubernetes metadata enrichment — see below.
OpenTelemetry	Add `dt.security_context` as a resource attribute in `OTEL_RESOURCE_ATTRIBUTES` or in the OpenTelemetry Collector.

2. Central configuration

Under **Settings > Collect and capture**, the central primary Grail tag configuration maps a tag, label or resource attribute to `dt.security_context` — or, per the docs, can *"Set a fixed value for all signals within a given scope (environment, Kubernetes cluster, host group)."* That fixed value is how you give a whole host group one security context; nothing assigns it automatically from group membership.

3. OpenPipeline (refinement)

Use OpenPipeline only where the value has to be computed from several fields: *"define dt.security_context in OpenPipeline using dedicated Set security context processors. These are available for spans, logs, and metrics."* Configure them in the OpenPipeline app for the relevant scope.

Kubernetes

Use existing namespace labels or annotations: *"Via the Kubernetes metadata enrichment feature, you can use already existing namespace labels and annotations as source for your security context."* Metadata enrichment must be enabled in Dynatrace Operator.

A pod annotation is the fallback, not the default — *"Dedicated pod annotations are only intended for scenarios where namespace labels or annotations cannot be used as a source."* The documented annotation is:

```
metadata:
  annotations:
    metadata.dynatrace.com/dt.security_context: "team-checkout"
```

It also enriches less: *"They do not enrich Kubernetes metrics, Kubernetes events, Kubernetes Smartscape entities, or Prometheus metrics."*

> **Sources:** Configure security context (DT docs), Kubernetes security context (DT docs).

IAM Policies with Security Context

Basic Security Context Policy

```
{
  "name": "team-a-data-access",
  "description": "Team A can access data with team-a security context",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name
STARTSWITH 'default_'; ALLOW storage:logs:read WHERE
storage:dt.security_context = 'team-a';",
  "tags": ["team:team-a"]
}
```

Hierarchical Access with MATCH

Grant access to entire department:

```
{
  "name": "finance-department-access",
  "description": "Finance department can access all finance sub-teams",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name
STARTSWITH 'default_'; ALLOW storage:logs:read WHERE
storage:dt.security_context MATCH ('finance/*');",
  "tags": ["department:finance"]
}
```

Array Field Access with MATCH

> **Important**: When `dt.security_context` holds an array, you MUST use `MATCH` operator. Using `=`, `STARTSWITH`, or `IN` will always return false for array fields.

```
// Correct - MATCH for array fields
ALLOW storage:logs:read WHERE storage:dt.security_context MATCH ("crn-70400-
*");

// Wrong - = won't work for arrays
ALLOW storage:logs:read WHERE storage:dt.security_context = "crn-70400-team";
```

> **Sources:** Permissions in Grail (DT docs) — *"Using =, STARTSWITH or IN when the field holds an array will always return false."*

Querying Security Context

DQL: Security Context Verification

Verify that `dt.security_context` is being set on your log data:

```
// Summarize log volume by security context — verify dt.security_context is
assigned correctly
fetch logs, from:-1h
| filter isNotNull(dt.security_context)
| summarize recordCount = count(), by:{dt.security_context}
| sort recordCount desc
| limit 20
```

Security Context Patterns

Pattern 1: Team-Based Access

```
Context value: "team:&lt;team-name&gt;"
Examples: "team:platform", "team:checkout", "team:payments"
Policy: ALLOW ... WHERE storage:dt.security_context = "team:platform"
```

Pattern 2: Line of Business

```
Context value: "lob:&lt;business-unit&gt;"
Examples: "lob:finance", "lob:retail", "lob:healthcare"
Policy: ALLOW ... WHERE storage:dt.security_context = "lob:finance"
```

Pattern 3: Hierarchical Organization

```
Context value: "&lt;org&gt;/&lt;department&gt;/&lt;team&gt;"
Examples: "acme/engineering/platform", "acme/finance/compliance"

Department-level access:
ALLOW ... WHERE storage:dt.security_context MATCH ("acme/engineering/*")

Team-level access:
ALLOW ... WHERE storage:dt.security_context = "acme/engineering/platform"
```

> `=` **works only while the field holds a single string.** Patterns 1–3 use `=` for exact values. *Permissions in Grail* is explicit: *"Using =, STARTSWITH or IN when the field holds an array will always return false. If you expect your record filters might contain an array, use the MATCH operator in your IAM statements."* Check the shape of your

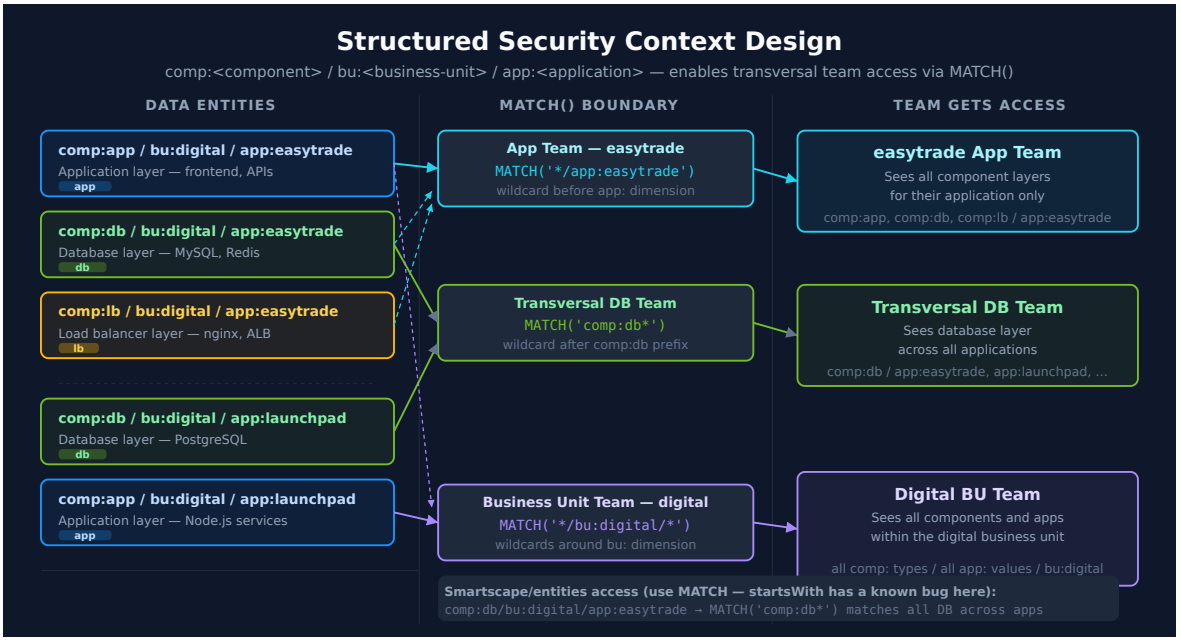
> data first (the shape-check query in ORGNZ-99 §5). If any record can carry an array, write the exact value with `MATCH`, which needs no wildcard:
>
> `> ALLOW storage:logs:read WHERE storage:dt.security_context MATCH ("team:platform"); > ALLOW storage:logs:read WHERE storage:dt.security_context MATCH ("lob:finance"); > ALLOW storage:logs:read WHERE storage:dt.security_context MATCH ("acme/engineering/platform"); >`
>
> **Sources:** Permissions in Grail (DT docs) — as quoted; the page's own team example is `MATCH ("TeamA")`.

Pattern 4: Cloud Account Mapping

```
Context value: "&lt;cloud&gt;/&lt;account&gt;"
Examples: "aws/123456789012", "gcp/my-project-id"
Policy: ALLOW ... WHERE storage:dt.security_context MATCH ("aws/*")
```

Pattern 5: Multi-Dimensional Structured Context

Encode multiple dimensions into a single string for transversal team access — teams that need cross-application visibility (database, networking, OS) without being tied to a specific application.



```
Context format: "comp:&lt;component&gt;/bu:&lt;business-
unit&gt;/app:&lt;application&gt;"
Examples:
  "comp:app/bu:digital/app:easytrade"  — application layer, digital BU
  "comp:db/bu:digital/app:easytrade"   — database layer, digital BU
  "comp:lb/bu:digital/app:easytrade"   — load balancer layer, digital BU
```

MATCH() access patterns (Grail / 3rd Gen storage domain):

Team Type	MATCH() Expression	Grants Access To
App team	`MATCH('*/app:easytrade')`	All components for one application
Transversal DB team	`MATCH('comp:db*')`	Database layer across all applications
Business unit team	`MATCH('*/bu:digital/*')`	All components and apps within digital BU

startsWith patterns — these are Grail `storage:` statements, so the array caveat under Pattern 3 applies: `STARTSWITH` returns false on an array-valued field. Use them only where every record holds one string; otherwise use the `MATCH()` table above. (Classic entity access uses the same prefix idea; see IAM-05.)

```
// App team — enumerate each component explicitly
ALLOW storage:logs:read WHERE storage:dt.security_context startsWith
"comp:app/bu:digital/app:easytrade"
ALLOW storage:logs:read WHERE storage:dt.security_context startsWith
"comp:db/bu:digital/app:easytrade"

// Transversal DB team — one prefix covers all apps
ALLOW storage:logs:read WHERE storage:dt.security_context startsWith "comp:db"
```

> **Dimension order matters for Classic entities.** Place the dimension used for transversal access *first* (typically `comp`). For the full two-domain comparison and multi-value context roadmap, see **IAM-04: Policy Authoring** and **IAM-05: Boundary Design**.

Security Context Best Practices

Practice	Rationale
Use consistent naming conventions	Enables pattern matching in policies
Plan hierarchy before implementation	Hard to change after data is ingested
Use prefixes for context types	`team:`, `lob:`, `env:` for clarity
Document context values	Governance and audit requirements
Test with sample data	Verify context is assigned correctly
Use MATCH for array fields	Required for correct policy evaluation
Encode multiple dimensions for transversal access	`comp/bu/app` format scales to cross-application teams without per-app policies

Entity Security Context

Security context can also be set on entities (not just records):

1. Go to **Settings** > **Topology model** > **Grail Security Context**
2. Define rules to assign security context to entities
3. This scopes access to the **entities** only. Records sent from those entities are not scoped by it: *"Logs, spans, metrics, and events powered by Grail that are sent from an entity do not inherit the management zones of that entity."* To scope a host's records, set `dt.security_context` as a host tag (see §3) — *"After you set the security context on a host, it will be used to automatically determine the security context for all logs, spans, metrics, and events that are sent from this host."*

This enables entity-level access control for services, hosts, and other monitored entities.

> **Sources:** Grant access to entities with security context (DT docs).

Next Steps

Continue with the ORGNZ series:
- **ORGNZ-07**: Advanced Permission Patterns

References

- Configure advanced permissions with security context
- Set up Grail permissions for OneAgent
- Grant access to entities with security context

This notebook was AI-generated from Dynatrace documentation and enterprise best practices. It is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.